

Spezifikation Universal Control Room Interface (UCRI2)

Version 2.0.0

Inhaltsverzeichnis

1. [Einleitung](#)
2. [Systemarchitektur](#)
 - 2.1. [Architekturstil Messaging](#)
 - 2.2. [Vermittlungsebene](#)
 - 2.3. [Anwendungsebene](#)
 - 2.4. [Sicherheitskonzept](#)
 - 2.5. [Adressierungskonzept](#)
 - 2.5.1. [Beispielhafte OID-Hierarchie](#)
 - 2.5.2. [Beispielhafte OID-Nomenklatur](#)
 - 2.6. [Versionierung](#)
 - 2.6.1. [Transportschicht-Versionierung](#)
 - 2.6.2. [App-Versionierung](#)
 - 2.6.3. [Spezifikations-Bundle \(BOM\)](#)
3. [Kommunikationsprotokoll](#)
 - 3.1. [Übermittlung von Nachrichten](#)
 - 3.2. [Validierung von Nachrichten](#)
 - 3.3. [KT-Register](#)
 - 3.4. [Datenintegrität](#)
 - 3.4.1. [Signierung](#)
 - 3.4.2. [Verschlüsselung](#)
4. [API](#)
 - 4.1. [Client-API](#)
 - 4.1.1. [/token - HTTP GET](#)
 - 4.1.2. [/info - HTTP GET](#)
 - 4.1.3. [/registry - HTTP GET](#)
 - 4.1.4. [/registry/{id} - HTTP GET](#)
 - 4.1.5. [/messaging/send - HTTP POST](#)
 - 4.1.6. [/messaging/receive - HTTP POST](#)
 - 4.1.7. [/messaging/commit - HTTP POST](#)
 - 4.2. [Peer-to-Peer-API \(P2P-API\)](#)
 - 4.2.1. [/token - HTTP GET](#)
 - 4.2.2. [/info - HTTP GET](#)
 - 4.2.3. [/registry - HTTP GET](#)
 - 4.2.4. [/messaging/send - HTTP POST](#)
 - 4.3. [Fehlerbehandlung](#)
 - 4.3.1. [Fehlercodes für Antworten mit HTTP-Statuscode 400](#)
 - 4.3.2. [Fehlercodes für Antworten mit HTTP-Statuscode 401](#)
 - 4.3.3. [Fehlercodes für Antworten mit HTTP-Statuscode 500](#)
5. [Applikationen](#)
6. [Systemintegration](#)
 - 6.1. [Integrationsmuster](#)
 - 6.2. [Integrationsszenarien](#)

1. Einleitung

UCRI steht für Universal Control Room Interface – ein Protokoll für die Kommunikation zwischen zwei oder mehreren Einsatzleitsystemen.

Nach der erfolgreichen Einführung und eingehenden Erprobung in der Praxis wurden Erfahrungen gesammelt und neue Anforderungen identifiziert. Diese machten eine Weiterentwicklung des UCRI-Protokolls auf einer neuen Architekturbasis erforderlich. UCRI2 ist eine vollständig überarbeitete Version, die alle Anwendungsfälle der Vorgängerversion (UCRI 1.1) unterstützt und die Grundlage für zukünftige, flexible Erweiterungen bildet.

Die Ziele für die Entwicklung von UCRI2 sind:

- Einfache, schnell zu implementierende API
 - standardisierte maschinenlesbare Spezifikation
 - sichere Zustellung und Verarbeitung von Meldungen
- Trennung technischer und fachlicher Aspekte
 - Einfache Erweiterbarkeit
 - Technische Komponenten müssen bei fachlichen Erweiterungen nicht angepasst werden
 - Fachliche Erweiterungen können ohne Anpassungen der Infrastruktur erfolgen
- Messaging-basierte Architektur
 - Einfaches Zusammenspiel mehrerer Hersteller
 - Unterstützung zentraler als dezentraler Anbindungen
- Durchgängige Standardisierung
 - BSI-konformes Netzarchitektur und -design
 - OpenAPI-Spezifikationen für Client- und P2P-APIs
 - JSON Schema Spezifikationen für Anwendungen (UCRI2-Apps) zur Nachrichtenvalidierung
 - OAuth 2.0 für Authentifizierung und Autorisierung
 - RFC 3447 – Kryptographische Verfahren
 - RFC 8785 – JSON Canonicalization Scheme (JCS)
 - RFC 7517 – JSON Web Key

Die vorliegende Spezifikation ist in folgende Kapiteln gegliedert:

Systemarchitektur

In diesem Kapitel wird UCRI2 als eine n:m-Kommunikationsarchitektur zwischen verschiedenen Kommunikationsteilnehmern unter Verwendung des Messaging-Architekturstils beschrieben. Die Trennung zwischen der fachlichen Anwendungsebene und der technischen Vermittlungsebene sowie ein umfassendes Sicherheitskonzept wird eingeführt.

Kommunikationsprotokoll

Das UCRI2-Kommunikationsprotokoll definiert zwei Schnittstellen: die Client-API für die Kommunikation zwischen einem Leitstellensystem und der UCRI2-Infrastruktur sowie die P2P-API für die Kommunikation zwischen verteilten Komponenten der UCRI2-Infrastruktur basierend auf Peer-to-Peer-Topologie. In diesem Kapitel werden Mechanismen für Nachrichtenübermittlung und -validierung sowie zur Gewährleistung der Datenintegrität durch kryptografische Verfahren ausführlich beschrieben.

API

Die UCRI2-APIs werden als REST-Services entworfen und mit OpenAPI 3.1.0-Spezifikationen dokumentiert. In diesem Kapitel werden die Gliederung der APIs in Funktionsbereiche sowie die Semantik der einzelnen API-Endpunkte

beschrieben.

Applikationen

Die Beschreibung der UCRI2-Applikationen wird in Form von JSON-Schemata samt begleitender Dokumentation separat zur Verfügung gestellt.

Systemintegration

Ein wichtiges Ziel der UCRI2-Spezifikation ist es, Interoperabilität in komplexen IT-Landschaften sicherzustellen. In diesem Kapitel wird anhand von verschiedenen Integrationsszenarien erläutert, wie konkrete Projektanforderungen mit Hilfe von spezialisierten UCRI2-Softwarekomponenten umgesetzt werden können.

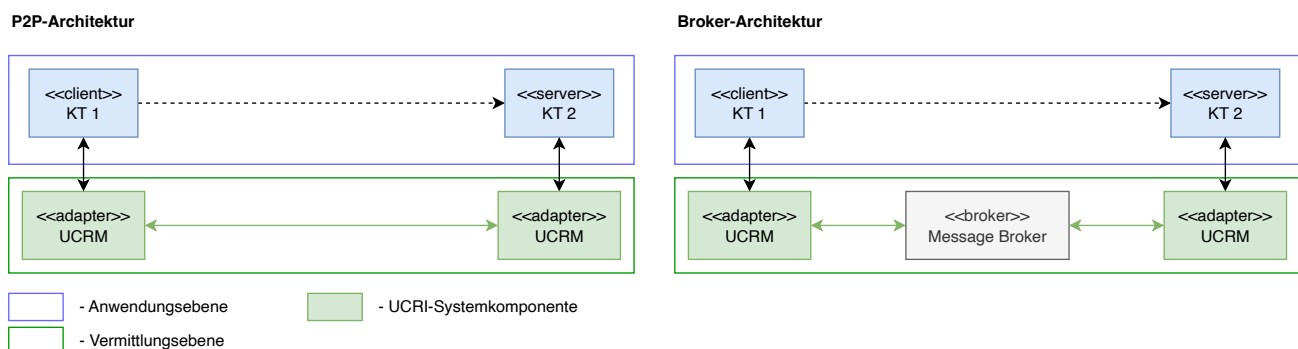
2. Systemarchitektur

Im Gegensatz zu UCRI Version 1, welche 1:1-Kommunikation zwischen Leitstellen spezifiziert, ist UCRI 2 grundlegend für die n:m-Kommunikation verschiedener Teilnehmer entwickelt worden.

2.1. Architekturstil Messaging

Bei der Strukturierung der UCRI2-Schnittstelle wird der Architekturstil Messaging verwendet. Bei diesem Architekturstil kommunizieren verteilte unabhängige Systemkomponenten (im allgemeinen Kommunikationsteilnehmer – kurz KT genannt) miteinander mit Hilfe von Nachrichten.

Messaging-Systeme trennen die fachliche Anwendung (gewöhnlich nach dem Client-Server-Prinzip strukturiert) von der Vermittlungsebene – also von technischen Aspekten der Nachrichtenübermittlung zwischen technischen Systemen der KT. Nachrichten können während der Übertragung umgewandelt werden, ohne dass Sender oder Empfänger von der Umwandlung wissen. Die Entkopplung ermöglicht es Integratoren, je nach Anforderung unterschiedliche Kommunikationstopologien zu implementieren, von dezentralen P2P-Protokollen bis zu zentralisierten Broker-Architekturen:



Messaging-Systeme ermöglichen es den Komponenten, entkoppelt zu bleiben und sich auf ihre eigenen Aufgaben zu konzentrieren, während sie gleichzeitig in der Lage sind, mit anderen Komponenten im System zu kommunizieren und zusammenzuarbeiten. Es verringert die Anzahl der Abhängigkeiten zwischen den Komponenten, wodurch das System flexibler und leichter zu warten ist.

2.2. Vermittlungsebene

Das andere wichtige Architekturmuster, das bei der Strukturierung der UCRI2-Schnittstelle Verwendung findet, ist das Adapter-Muster. Bei diesem Muster erfolgt die Kommunikation zwischen dem technischen System der KT

(Anwendungsebene) und der Vermittlungsebene mittels einer Adapter-Komponente (UCRI Control Room Module - UCRM). Der Adapter ermöglicht bidirektionale Kommunikation zwischen den Ebenen in einer standardisierten Form und ermöglicht die Komplexitätsreduzierung der angebundenen Schnittstellen.

Das UCRM stellt die UCRI2-Client-API bereit - die einzige Kommunikationsschnittstelle für direkt verbundene Kommunikationsteilnehmer wie Leitstellensysteme oder andere technische Knoten, sowie weitere externe Systeme.

Die untereinander direkt oder über einen Broker kommunizierenden UCRM-Adapter bilden die Vermittlungsebene und kümmern sich somit um die technischen Aspekte der Kommunikation, während sich die KT's auf die eigentliche Anwendungslogik fokussieren - also die Anwendungsebene.

Zentrale Aufgabe der Vermittlungsebene ist die Zustellung von Meldungen zwischen Sender und Empfänger. Außerdem werden auf der Vermittlungsebene unterschiedliche Querschnittsaufgaben übernommen.

Einzelne Aufgaben der Vermittlungsebene sind:

- Verwaltung der Kommunikationstopologie inklusive Adressierungskonzept, KT-Status-Monitoring und KT-Register
- Authentisierung, Autorisierung, Accounting
- Übermittlung von Nachrichten unter der Verwendung des UCRI-Adressierungskonzepts
- Validierung von Anwendungsmeldungen

Die Vermittlungsebene ist frei von Fachlichkeit. Sie realisiert nur den Datentransport und sichert die Integrität der Daten.

2.3. Anwendungsebene

Die UCRI2-Anwendungsebene ist grundsätzlich getrennt von der Vermittlungsebene. Die Anwendungsebene ist dabei in mehrere unabhängige Applikationen, sogenannte UCRI2-Apps, aufgeteilt.

Eine UCRI2-Applikation wird durch folgende Artefakte definiert:

- Ein Satz von standardisierten Nachrichten-Schemata (JSON, kanonisches Datenmodell)
- Ablaufmodell (definierte Abfolge von Nachrichten)
- Prozessdefinitionen (Festlegungen bezüglich Anwendungslogik, die bei der Implementierung in technischen KT-Systemen berücksichtigt werden müssen)

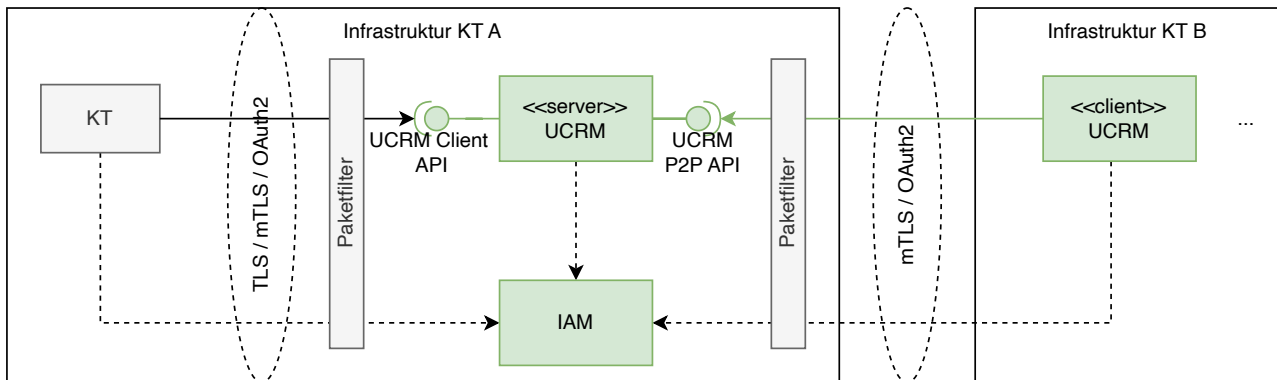
Einzelne Applikationen werden separat und unabhängig von der Spezifikation der Vermittlungsebene (Transportschichtspezifikation) versioniert (siehe [UCRI2 Versionierung](#)). Dies erlaubt die freie Weiterentwicklung jeder einzelnen UCRI2-App.

2.4. Sicherheitskonzept

Einzelne Sicherheitsaspekte werden im Folgenden im Hinblick auf die Standard-UCRI2-Kommunikationsarchitektur betrachtet. Spezifische Systemlösungen (siehe Kapitel [Systemintegration](#)) sollen entsprechende projektspezifische Sicherheitsanforderungen berücksichtigen.

UCRI2 unterscheidet zwei Kommunikationsdomänen, siehe Abbildung:

- Kommunikation zwischen einem Leitstellensystem (KT-System) und einem UCRI Leitstellenmodul (UCRM). Diese Kommunikation findet anhand der UCRI2 Client-API statt und erfolgt typischerweise innerhalb einer durch einen Leitstellenbetreiber kontrollierten Infrastruktur.
- Kommunikation zwischen zwei UCRM. Die Inter-UCRM-Kommunikation in einer P2P-Architektur verwendet die UCRI2 P2P-API. Die Inter-UCRM-Kommunikation kann über das öffentliche Internet stattfinden und setzt in diesem Falle besondere Sicherheitsmaßnahmen voraus.

P2P-Architektur

Die Infrastruktur eines KT sollte eine P-A-P-Struktur aufweisen, die aus Paketfilter als Trennung zu vertrauenswürdigen internen Systemen (Leitstelle), Application-Layer-Gateway (UCRM) und Paketfilter als Trennung zu nicht vertrauenswürdigen Netz (Internet) besteht (Vgl. [Netzarchitektur und -design - BSI](#)).

Beide API-Implementierungen verwenden in beiden Kommunikationsdomänen OAuth 2.0 mit Client Credentials Grant Type zur sicheren Authentifizierung und Autorisierung von Clients auf der Applikationsebene.

Trust Konzept

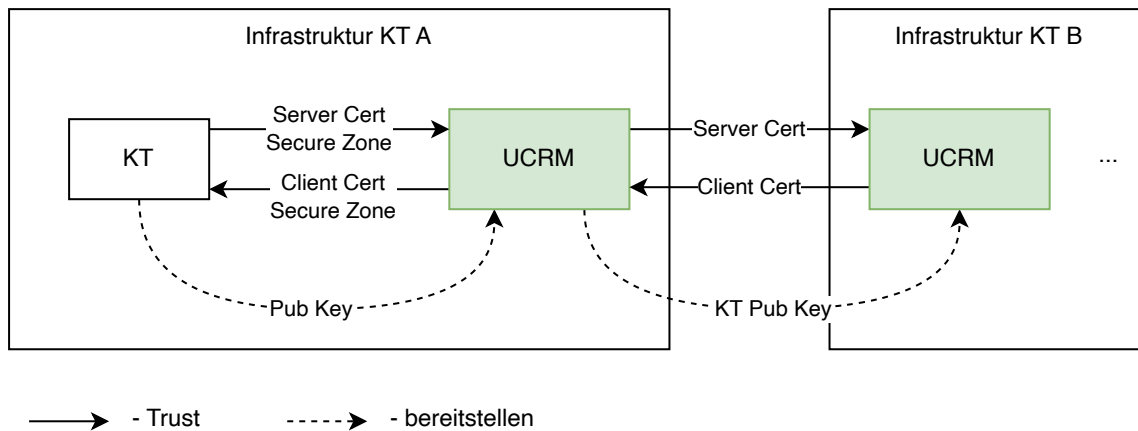
Bei einer P2P-Architektur wird mTLS als Sicherung der Inter-UCRM-Kommunikation zwischen zwei UCRM verwendet. Sowohl Client als auch Server bauen eine gegenseitige Vertrauensbeziehung über digitale Zertifikate auf. Dabei kann der Paketfilter auf der Seite des nicht vertrauenswürdigen Netzes die mTLS-Verbindung terminieren. Im Paketfilter kann eine Liste von zugelassenen Domainnamen implementiert (White Listing) und somit die Kommunikation auf eine geschlossene Gruppe der Teilnehmer beschränkt werden.

Die Kommunikation zwischen einem KT-System und dem UCRM kann je nach lokalen Sicherheitsanforderungen wahlweise per TLS oder mTLS erfolgen.

Die Vertrauensbeziehungen zwischen den Systemkomponenten bei der Client/Server-Kommunikation basieren dabei auf folgenden Mechanismen (siehe Abbildung):

1. Server-Authentifizierung. Die Vertrauensbeziehung zum UCRM als Server basiert auf dem Domainnamen (DNS) des Servers und wird durch ein Server-Zertifikat abgesichert. Der Domainname des Servers ist im Server-Zertifikat verankert.
2. Client-Authentifizierung. Die Vertrauensbeziehung zum KT oder UCRM in der Client-Rolle basiert auf dem Object Identifier (OID) des Clients (siehe [Adressierungskonzept](#)) und wird durch Client-Zertifikat abgesichert. Die OID des Clients ist im Client-Zertifikat verankert.
3. Sowohl Client als auch Server haben eine Liste vertrauenswürdiger Zertifizierungsstellen (CAs), sogenannte Root-CAs, die als Vertrauensanker dienen. Die ausgestellten Zertifikate von Client und Server müssen von einer CA signiert sein, die jeweils in der vertrauenswürdigen Liste des Gegenübers enthalten ist.

P2P-Architektur



Anmerkung 1: In den konkreten Kundensituationen können unterschiedliche Mechanismen des Zertifikatsmanagements umgesetzt werden – je nach Anforderung entweder auf zentraler Authority oder auf spezifischen Vereinbarungen zwischen einzelnen KTs.

Anmerkung 2: Als Alternative kann das Vertrauen zwischen KT-Systemen und dem UCRM in einer durch einen Leitstellenbetreiber kontrollierten Infrastruktur durch das Bilden einer Sicherheitszone auf der Netzwerkebene hergestellt werden.

Um den sicheren Ursprung und unverfälschten Inhalt von Applikationsmeldungen zu garantieren, können die Meldungen durch Sender-KT signiert werden. Zur Signaturvalidierung erstellt der KT ein kryptografisches Schlüsselpaar und stellt sein Public Key dem lokalen UCRM-Modul über einen sicheren Kanal bereit. Bei der Inter-UCRM-Kommunikation wird der Public Key des KT an die fremden UCRM mittels UCRM P2P-API übermittelt und steht anschließend den potenziellen Meldungsempfängern über die Client-API zur Signaturvalidierung zur Verfügung. Das Signieren von Nachrichten ist im Kapitel [Kommunikationsprotokoll](#) detailliert beschrieben.

2.5. Adressierungskonzept

Für die Adressierung der einzelnen Kommunikationsteilnehmer (KT) auf der Vermittlungsebene werden eindeutige Kennungen gemäß dem Object Identifier Standard (OID Spezifikationen ISO/IEC 9834, DIN 66334) verwendet. Für die Festlegung von OIDs gelten folgende Vorgaben:

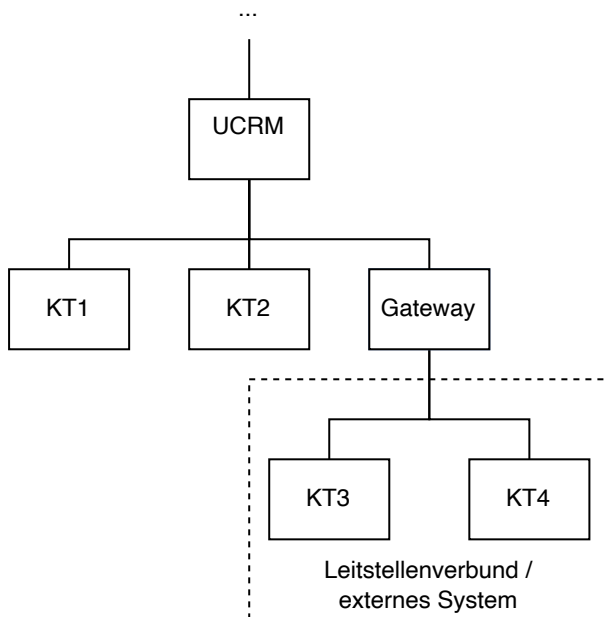
1. Falls möglich, sollten offiziell zugeteilte OIDs zum Einsatz kommen. Diese können z.B. für das deutsche Gesundheitswesen beim Bundesinstitut für Arzneimittel und Medizinprodukte beantragt werden.
2. Falls keine offiziell vergebenen OIDs zum Einsatz kommen, dürfen selbst vergebene OIDs NIEMALS in einem Adressraum liegen, in dem auch offiziell zugeteilte OIDs vergeben werden, um Überschneidungen mit offiziell zugeteilten OIDs zu vermeiden.

Zusätzlich wird empfohlen, die genutzten OIDs dem Expertenforum UCRI zu melden, damit Adresskonflikte direkt bei der Vergabe vermieden werden können.

Die in den folgenden Unterkapiteln vorgestellten beispielhaften OID-Hierarchien und OID-Nomenklatur stellen nur eine Empfehlung dar. Falls von offiziellen Stellen vergebene OIDs zum Einsatz kommen, entsteht nicht zwangsläufig auch eine Hierarchie, wie sie in den folgenden Unterkapiteln dargestellt wird.

2.5.1. Beispielhafte OID-Hierarchie

Die Adressierung von einzelnen KT kann hierarchisch organisiert werden und spiegelt dann die hierarchische Kommunikationsstruktur wider:



Die Systemkomponente Gateway stellt einen speziellen KT dar. Das Gateway wird am Übergang zu externen Systemen eingesetzt und stellt eine Funktion zum Mapping zwischen externe Quell- bzw. Zieladressen und internen OID bereit. Ein externes System bekommt dabei einen entsprechend reservierten OID-Bereich (Unterbaum) und das Gateway bekommt die Wurzel-OID-Adresse dieses Unterbaums.

2.5.2. Beispielhafte OID-Nomenklatur

Alle Kommunikationsteilnehmer in einem UCRI-System bekommen eine in diesem System eindeutige OID zugewiesen. Dieser OID-Adressraum wird durch eine Wurzeladresse (im weiteren Verlauf als bezeichnet) festgelegt. Die Strukturierung des UCRI-OID-Adressierungsraums (UCRI-OID-Nomenklatur) ermöglicht eine Adressierung von KT über die staatlichen Grenzen hinaus.

Dabei steht jedem Land frei, ein eigenes Adressierungsschema unterhalb des Landes-OID-Unterbaums zu definieren. Die Wurzeladresse eines Landes-OID-Unterbaums ist wie folgt definiert:

- <Root-OID>.1.<Kennzahl des Landes> (Beispiel: .1.276 für Deutschland)

Auch jedes Bundesland in Deutschland ist frei, ein eigenes Adressierungsschema unterhalb des Bundesland-OID-Unterbaums zu definieren. Wurzel-Adresse eines Bundesland-OID-Unterbaums ist wie folgt definiert:

- <Root-OID>.1.276.<Kennzahl des Bundeslandes> (Beispiel: .1.276.5 für NRW)

Unterschiedliche Organisationen können auch unterschiedliche Adressierungsschemata verwenden. Wurzel-Adresse eines nPOLGA-OID-Unterbaums ist wie folgt definiert:

- <Root-OID>.1.276.5.<Kennzahl von POL/nPOLGA, etc.> (Beispiel: .1.276.5.1 für nPOLGA in NRW)

Die OID-Adresse der einzelnen Kommunikationsteilnehmer (KT) wird nach dem [amtlichen Gemeindeschlüssel \(AGS\)](#) gebildet:

#	Ebene	Beispiel
---	-------	----------

1	Root-OID	1.2.3 - Festlegung in einem geschlossenen UCRI-System
2	Satzart	1 - Einzeladresse, 2 - Gruppe (aktuell werden in UCRI keine Gruppenadressen unterstützt)
3	Kennzahl des Landes	276 - Deutschland nach ISO 3166
4	Kennzahl des Bundeslandes	5 - für NRW nach AGS
5	Kennzeichnung von POL/nPOL Gefahrenabwehr (KRITIS, etc.)	Polizeidienststellen, Gesundheitsämter, Veterinärämter, etc. Definition folgt. Annahme für Beispiel: 1 - nPOLGA, 99 - Test/Pilot
6	Kennzahl des Regierungsbezirks	1 - Regierungsbezirk Düsseldorf nach AGS
7	Kennzahl des Landkreises oder der kreisfreien Stadt ("0" bei zentralen Einrichtungen auf der Regierungsbezirksebene)	58 - Landkreis Mettmann nach AGS
8	Gemeinde ("0" bei kreisfreien Städten)	28 - Stadt Ratingen, Stadtbezirk Ratingen nach AGS
9	Leitstellenmodul	1 - erstes Leitstellenmodul, fortlaufende Nummerierung von Leitstellenmodulen
10	Kommunikationsteilnehmer	1 - z.B. ELS, fortlaufende Nummerierung von KT

Beispiel OID Feuerwehr ELS in Ratingen: .1.276.5.1.1.58.28.1.1

2.6. Versionierung

Die Versionierung für die Vermittlungsebene (UCRI2-Transportschicht) und die UCRI2-Apps erfolgen voneinander getrennt. Beide Versionierungen folgen den Regeln von [Semantic Versioning 2.0.0](#).

Zusätzlich werden eine Transportschicht-Version und ein Satz konkreter App-Versionen zu einem [UCRI2-Spezifikations-Bundle](#) zusammengefasst, welches ebenfalls eigenständig versioniert wird.

Diese Spezifikation beschreibt die Version 2.0.0 der UCRI2-Transportschicht.

2.6.1. Transportschicht-Versionierung

Die Transportschichtspezifikation umfasst dieses Dokument sowie die OpenAPI-Spezifikationen für die Client- und die P2P-Schnittstellen.

Die Versionsnummer für die Transportschichtspezifikation besteht aus drei numerischen Teilen:

MAJOR.MINOR.PATCH

Für die drei Versionsbestandteile gelten folgende Festlegungen:

- **MAJOR** : Hauptversion der Transportschicht. Eine Änderung dieser Version erfolgt, wenn Änderungen an den API-Endpunktdefinitionen erfolgen, die die Kompatibilität brechen. Hierzu zählen insbesondere: Verändern bestehender Endpunkte bezüglich obligater Felder, Hinzufügen neuer obligater Felder, Entfernen von Endpunkten oder Feldern sowie das Verengen bestehender Wertebereiche (z.B. Entfernen eines Enum-Wertes, Verkleinern einer zulässigen Länge). Die **MAJOR** -Version bezeichnet zugleich die UCRI-Generation und ist für UCRI2 auf **2** festgelegt.

- **MINOR** : Unterversion der Transportschicht. Eine Änderung dieser Version erfolgt, wenn neue optionale Felder zu bestehenden Endpunkten hinzugefügt oder neue optionale Endpunkte hinzugefügt werden.
- **PATCH** : Korrekturversion der Transportschicht. Eine Änderung dieser Version erfolgt bei Änderungen ohne Auswirkung auf den Schnittstellenvertrag, also insbesondere bei redaktionellen Korrekturen, Präzisierungen von Beschreibungstexten sowie Korrekturen von Beispielen.

Somit sind Änderungen an der **MINOR** - und der **PATCH** -Version stets kompatibel, sodass Systeme mit übereinstimmender **MAJOR** -Version untereinander kommunizieren können, auch wenn sie unterschiedliche **MINOR** - oder **PATCH** -Versionen aufweisen.

Hinweis zur bisherigen Notation: In früheren Entwurfsfassungen wurde die Notation **GEN.MAJOR.MINOR** verwendet, bei der die erste Stelle (**GEN**) ausschließlich die UCRI-Generation bezeichnete. Diese Notation ist entfallen. Die Bedeutung der Stellen ist unverändert geblieben, die Generationsangabe geht in der **MAJOR** -Stelle auf. Die Versionsbezeichnung **2.0.0** ist in beiden Notationen identisch.

Vorabversionen

Für Fassungen, die sich noch in Abstimmung befinden und noch nicht zur Implementierung freigegeben sind, wird gemäß Semantic Versioning ein Vorabversions-Suffix verwendet, z.B. **2.1.0-rc.1** . Vorabversionen sind nicht Bestandteil eines freigegebenen Spezifikations-Bundles.

Transportschicht-Meldungen

Die Transportschicht verwendet eine UCRI2-App "Transportschicht-Meldungen" (`transport_layer_messages`). Diese App beschreibt Nachrichten, die auf der Transportschicht erstellt und von verbundenen UCRM sowie Clients konsumiert werden. Somit MUSS eine spezifische Version dieser App durch alle Clients sowie UCRM, welche die Version 2.0.0 der Transportschicht implementieren, ZWINGEND unterstützt werden.

Die hierfür zu unterstützende Version wird im [Spezifikations-Bundle](#) festgelegt und ist dort als obligatorisch gekennzeichnet.

2.6.2. App-Versionierung

Die App-Versionierung erfolgt für jede App individuell. Die Versionsnummer einer App besteht aus drei numerischen Teilen:

MAJOR.MINOR.PATCH

Für die drei Versionsbestandteile gelten folgende Festlegungen:

- **MAJOR** : Hauptversion der App. Eine Änderung dieser Version erfolgt bei allen Änderungen, die die Kompatibilität brechen. Hierzu zählen insbesondere: Hinzufügen neuer obligater Nachrichten, Hinzufügen oder Verändern obligater Felder in bestehenden Nachrichten, Entfernen von Nachrichten oder Feldern, das nachträgliche Erklären eines optionalen Feldes zu einem obligaten Feld sowie das Verengen bestehender Wertebereiche (z.B. Entfernen eines Enum-Wertes, Verkleinern einer zulässigen Länge, Verschärfen eines Musters).
- **MINOR** : Unterversion der App. Eine Änderung dieser Version erfolgt, wenn neue optionale Nachrichten hinzugefügt werden, in bestehenden Nachrichten optionale Felder hinzugefügt werden oder bestehende Wertebereiche erweitert werden.
- **PATCH** : Korrekturversion der App. Eine Änderung dieser Version erfolgt bei Änderungen ohne Auswirkung auf den Nachrichtenvertrag, also insbesondere bei redaktionellen Korrekturen, Präzisierungen von Titeln und Beschreibungen sowie Korrekturen von Beispielen.

Führung der **PATCH** -Stelle

Die **PATCH** -Stelle ist per Definition nicht interoperabilitätsrelevant. Sie wird daher NICHT geführt in:

- den Verzeichnisnamen der App-Schemata und der App-Dokumentation,

- den `$id` -Werten der JSON-Schemata,
- dem Feld `appVersion` der übertragenen Nachrichten,
- den im KT-Register publizierten App-Versionen.

An diesen Stellen wird eine App-Version stets in der verkürzten Form `MAJOR.MINOR` angegeben. Die vollständige Versionsangabe inklusive `PATCH` -Stelle wird ausschließlich im [Spezifikations-Bundle](#) sowie in der Änderungshistorie der jeweiligen App geführt. Das Spezifikations-Bundle führt zu jedem App-Eintrag zusätzlich eine eindeutige Quellreferenz (`sourceRef`), über die der exakte Schemastand auch bei gleichbleibender `MAJOR.MINOR` -Angabe eindeutig aufgelöst werden kann.

Bestandsschutz

Bereits veröffentlichte App-Versionen mit zweistelliger Angabe (z.B. `1.0`) gelten als `MAJOR.MINOR.0` (also `1.0.0`) im Sinne dieser Festlegungen.

Publikation im KT-Register

Das UCRI2-Protokoll beinhaltet ein KT-Register (siehe [KT-Register](#)), in dem App-Versionen, die ein Kommunikationsteilnehmer unterstützt, publiziert werden. Obwohl die Änderungen an der `MINOR` -Version stets kompatibel sind, müssen auch `MINOR` -Varianten unter den unterstützten Apps eines Teilnehmers explizit angegeben werden.

2.6.3. Spezifikations-Bundle (BOM)

Eine Transportschicht-Version allein beschreibt noch keinen vollständig interoperablen Systemstand, da die inhaltliche Kommunikation über die UCRI2-Apps erfolgt, welche unabhängig von der Transportschicht versioniert werden. Um einen prüfbaren und ausschreibungsfähigen Bezugspunkt zu schaffen, fasst UCRI2 eine Transportschicht-Version und einen Satz konkreter App-Versionen zu einem **Spezifikations-Bundle** zusammen.

Das Spezifikations-Bundle ist nach dem Vorbild einer Bill of Materials (BOM) aufgebaut und wird maschinenlesbar in der Datei [ucri2-bom.json](#) geführt. Die zugehörige Struktur ist in [ucri2-bom.schema.json](#) festgelegt.

Abgrenzung zur Transportschicht-Version

Die Bundle-Version ist von der Transportschicht-Version entkoppelt und wird eigenständig gezählt. Dies ist erforderlich, damit ein neuer App-Zusammenstellungsstand veröffentlicht werden kann, ohne die Transportschicht-Version zu verändern, obwohl an dieser keine Änderung erfolgt ist.

Aufbau der Bundle-Version

Die Bundle-Version folgt dem Schema `MAJOR.MINOR.PATCH` gemäß Semantic Versioning. Sie leitet sich aus den enthaltenen Artefakten ab:

- **MAJOR** : Die Bundle-Hauptversion wird erhöht, wenn die enthaltene Transportschicht-Version oder die Version mindestens einer enthaltenen App eine **MAJOR** -Erhöhung erfährt oder wenn eine App aus dem Bundle entfernt wird.
- **MINOR** : Die Bundle-Unterversion wird erhöht, wenn die enthaltene Transportschicht-Version oder die Version mindestens einer enthaltenen App eine **MINOR** -Erhöhung erfährt oder wenn eine App neu in das Bundle aufgenommen wird.
- **PATCH** : Die Bundle-Korrekturversion wird erhöht, wenn ausschließlich **PATCH** -Erhöhungen enthaltener Artefakte vorliegen.

Ergänzend führt jedes Bundle ein kalendarisches Label (`calendarLabel` , z.B. `UCRI2 Release 2026.1`). Dieses Label dient ausschließlich der Kommunikation, insbesondere gegenüber Beschaffung und Betrieb, und trägt KEINE Kompatibilitätsaussage. Maßgeblich für alle technischen Festlegungen ist stets die Bundle-Version.

Exakte Versionsangaben

Alle Versionsangaben in einem Spezifikations-Bundle MÜSSEN exakt angegeben werden. Versionsbereiche oder Platzhalter (z.B. `1.x` oder `^1.2`) sind NICHT zulässig. Nur so bleibt die Aussage "interoperabel gemäß UCRI2-Bundle X" eindeutig prüfbar.

Verbindlichkeit einzelner Einträge

Jeder App-Eintrag im Bundle ist über das Feld `mandatory` als obligatorisch oder optional gekennzeichnet:

- `mandatory: true` : Ein System, welches dieses Bundle implementiert, MUSS die App in der angegebenen Version unterstützen. Dies betrifft insbesondere die App "Transportschicht-Meldungen" (`transport_layer_messages`).
- `mandatory: false` : Die App ist Bestandteil des Bundles, ihre Unterstützung ist jedoch von der jeweiligen fachlichen Rolle des Systems abhängig. Welche Apps ein System tatsächlich unterstützt, wird über das KT-Register publiziert.

Apps außerhalb des Bundles

Kommunikationsteilnehmer DÜRFEN weitere Apps oder weitere Versionen der enthaltenen Apps unterstützen und im KT-Register publizieren. Eine solche Unterstützung ist jedoch nicht Bestandteil der durch das Bundle zugesicherten Interoperabilität.

Aktuelles Spezifikations-Bundle

Bundle-Version: `1.0.0` (UCRI2 Release 2026.1), Transportschicht-Version: `2.0.0`

App-ID	App-Version	Obligatorisch
<code>transport_layer_messages</code>	1.0.0	ja
<code>classification_catalogue</code>	1.0.0	nein
<code>incident_transfer</code>	1.0.0	nein
<code>incident_transfer_police</code>	1.0.0	nein
<code>incident_transfer_with_patient</code>	1.0.0	nein
<code>notification_text</code>	1.0.0	nein
<code>patient_transfer</code>	1.0.0	nein
<code>patient_transport</code>	1.0.0	nein
<code>resource_request</code>	1.0.0	nein
<code>resource_type_catalogue</code>	1.0.0	nein

Die Tabelle gibt den Inhalt der Datei [ucri2-bom.json](#) in lesbarer Form wieder. Im Konfliktfall ist die Datei maßgeblich.

3. Kommunikationsprotokoll

Im Folgenden werden einzelne Aspekte des UCRI2-Kommunikationsprotokolls im Hinblick auf die Standard-UCRI2-Kommunikationsarchitektur betrachtet. Hersteller von spezialisierten UCRM-Modulen (siehe Kapitel [Systemintegration](#)) müssen entsprechende spezifische Kommunikationsprotokolle umsetzen.

Aus der Sicht einer Leitstelle (KT-System) ist UCRI2 eine Client/Server Kommunikation gegen die UCRI2-Systemkomponente Leitstellenmodul (UCRM). Ein KT-System (Client) baut eine Verbindung zum UCRM (Server) auf und konsumiert die sogenannte **UCRI2-Client-API**. Die UCRI2-Client-API ist somit die einzige Schnittstelle, gegen die eine Leitstellensoftware integriert werden muss, um mit anderen Leitstellen über UCRI2 kommunizieren zu können.

Auf der Vermittlungsebene kommunizieren alle UCRM direkt miteinander. Jedes UCRM muss eine direkte Netzwerkverbindung zu jedem einzelnen Partner-UCRM aufbauen. Gleichzeitig bietet jedes UCRM eine Schnittstelle an – die sogenannte **UCRI2-P2P-API**, die von anderen Partner-UCRM konsumiert wird.

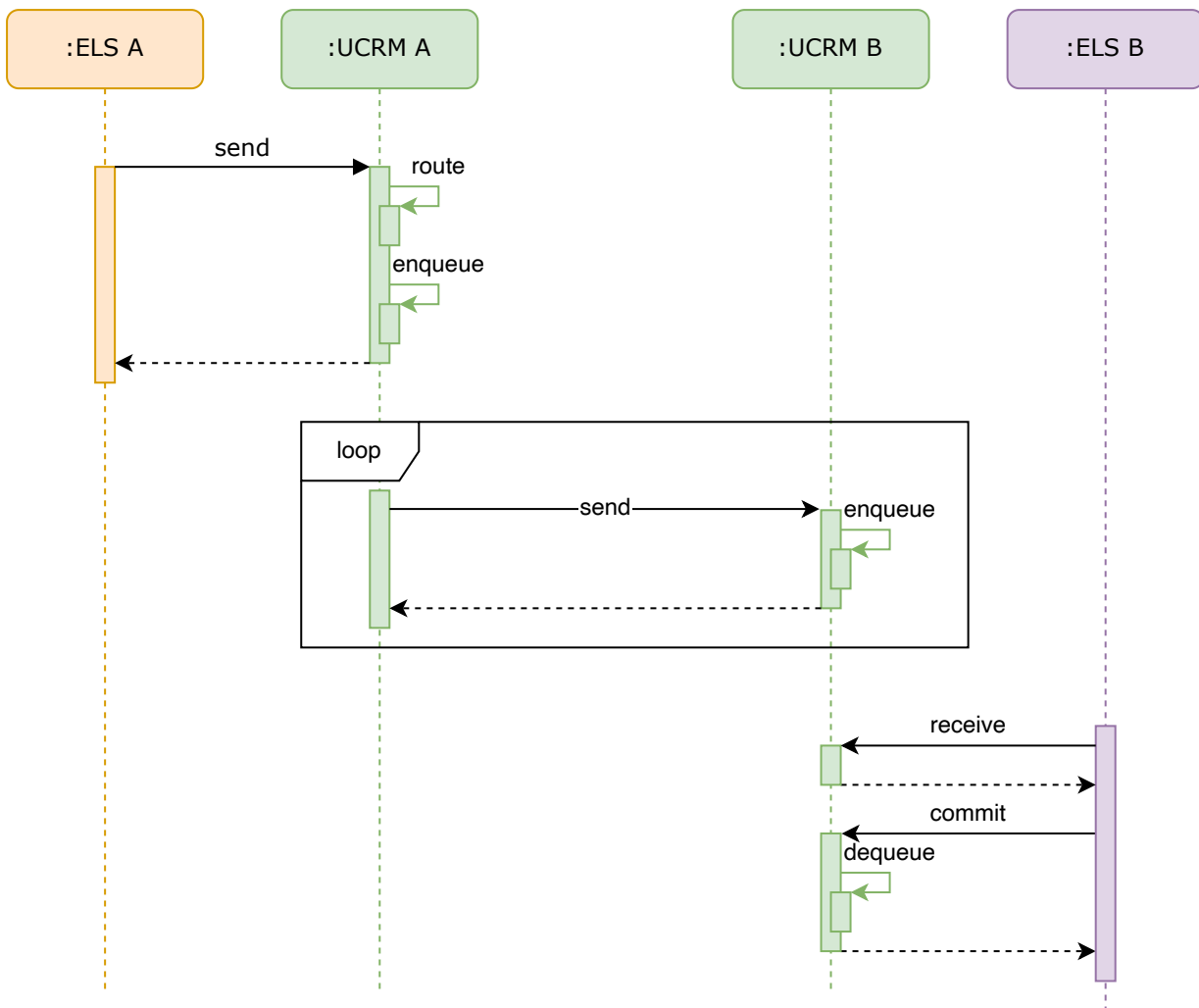
Je nach Rolle in der Gesamtkommunikation ergibt sich damit folgende Relevanz der vorliegenden Spezifikation für die beteiligten Systemkomponenten:

- Ein KT, der sich per UCRI2 mit anderen KT verbinden will, muss ausschliesslich die UCRI2 Client-API aus der Consumer-Perspektive umsetzen.
- Ein UCRM muss die UCRI2 Client-API aus der Provider-Perspektive umsetzen. Außerdem muss ein UCRM, das an einem Kommunikationssystem mit P2P-Topologie beteiligt ist, die UCRI2 P2P-API umsetzen (Provider- und Consumer-Perspektiven).

Die REST APIs Client-API und P2P-API werden [im Kapitel API](#) detailliert beschrieben.

3.1. Übermittlung von Nachrichten

Ein KT-System übergibt die ausgehenden Nachrichten an das UCRM-Modul mittels des Client-API-Endpunkts **/send**. Die Nachrichten werden dann durch das lokale UCRM mittels des P2P-API-Endpunktes **/send** an das Empfänger-UCRM weitergesendet. Für das Handling von Nichtverfügbarkeit der Partner-UCRMs wird in dem lokalen UCRM ein Ausgangspuffer implementiert.



3.2. Validierung von Nachrichten

Nachrichten werden in UCRM mittels App-spezifischer Nachrichtenschemata validiert. Die Validierung erfolgt synchron beim Senden. Das KT-Register liefert Auskunft über die durch KT unterstützten Applikationsschemata.

3.3. KT-Register

Zur Verwaltung der KTs dient ein KT-Register mit Systeminformationen (Systemname, Betreiber, technischer Support, etc.), Systemstatus, sowie Informationen über die unterstützten UCRI2-Anwendungen eines Kommunikationsteilnehmers. Das KT-Register wird durch das UCRM bereitgestellt und aktuell gehalten. Über die Client-API steht das KT-Register den angeschlossenen KT-Systemen zur Verfügung.

Jedes UCRM baut einen lokalen Cache des KT-Registers durch regelmäßige Aufrufe des P2P-API-Endpunktes **/registry** an allen bekannten Partner-UCRM auf. Dabei liefert ein UCRM nur eigene direkt angeschlossene KTs.

Ein angeschlossenes KT-System fragt das KT-Register über den Client-API-Endpunkt **/registry** ab. Dabei liefert ein UCRM alle ihm bekannten (sowohl eigene, als auch über Inter-UCRM-Kommunikation ermittelten) KTs.

Jedes UCRM besitzt einen eigenen Eintrag im KT-Register.

3.4. Datenintegrität

Die Kontrolle der Datenintegrität in Kommunikationssystemen ist von entscheidender Bedeutung, um sicherzustellen, dass die übertragenen Informationen vom richtigen Sender stammen und auf dem Übertragungsweg nicht manipuliert worden sind. Durch kryptografische Verfahren wie die **Signierung** wird die Authentizität und Unveränderbarkeit der Daten sichergestellt, während die **Verschlüsselung** die Vertraulichkeit gewährleistet und den Inhalt vor unbefugtem Zugriff schützt.

Als Richtlinie für die Auswahl kryptografischer Verfahren für die Signierung und Verschlüsselung von Nachrichten dient die entsprechende technische Richtlinie des BSI

(https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/TechnischeRichtlinien/TR02102/BSI-TR-02102.pdf?__blob=publicationFile&v=9).

Das asymmetrische kryptografische RSA-Verfahren RSASSA-PKCS1-v1_5 ([RFC 3447: Public-Key Cryptography Standards \(PKCS\) #1: RSA Cryptography Specifications Version 2.1](#)) wird sowohl zum Verschlüsseln als auch zum digitalen Signieren der Nachrichten verwendet.

Das Verfahren verwendet ein für jeden KT generiertes Schlüsselpaar, bestehend aus einem privaten Schlüssel und einem öffentlichen Schlüssel. Der private Schlüssel wird im Keystore des KT-Systems gespeichert.

Die öffentlichen Schlüssel werden über das KT-Register als Teil der verfügbaren KT-Daten in Form eines JSON Web Key (JWK, [RFC 7517: JSON Web Key \(JWK\)](#)) ausgetauscht und zur Verfügung gestellt.

3.4.1. Signierung

Nachrichtensignaturen stellen sicher, dass eine Nachricht während der Übertragung nicht verändert wurde und dass sie von der angegebenen Quelle stammt. Hierzu sind zwei Schritte notwendig: Die Erzeugung eines eindeutigen Hash-Wertes für die Nachricht sowie die Signierung dieses Hashwertes durch das sendende System:

- Für das Hashing wird die Nachricht zuerst gemäß JSON Canonicalization Scheme (JCS, [RFC 8785: JSON Canonicalization Scheme \(JCS\)](#)) in eine kanonische Form gebracht und diese kanonische Form dann mit SHA3-256 gehasht.
- Für das Signieren und die Prüfung der Signatur wird das Verfahren nach dem IETF-Standard JSON Web Signature (JWS, [RFC 7517: JSON Web Key \(JWK\)](#)) in der Variante Compact JWS in Kombination mit dem RSA-Verfahren RSASSA-PKCS1-v1_5 verwendet.

Diese Schritte werden im Folgenden genauer dargestellt.

Hashing der Nachrichten

Vor der Anwendung des JCS erstellt das signierende System eine Kopie der Nachricht, in der nur die folgenden Felder enthalten sind:

- "source"
- "destinations"
- "payload"

Somit werden alle anderen Felder nicht beim Hashing berücksichtigt. Dies ist notwendig, da andere Felder freiwillig sind oder vom UCRM gesetzt werden, falls sie nicht vom Client gesetzt wurden. Als Hashing-Algorithmus kommt SHA3-256 zum Einsatz ([Use of the SHA3 One-way Hash Functions in the Cryptographic Message Syntax \(CMS\)](#)).

Signierung der Nachrichten

Als JWS-Header kommt ein Header mit Angabe des RSA-256-Algorithmus zum Einsatz:

```
{
  "typ": "UCRI_PLAIN",
  "alg": "RS256"
}
```

Der berechnete Hashwert der Nachricht wird als JWS-Payload verwendet. Dieser wird mit dem privaten Schlüssel des Nachrichtensenders signiert. Das Ergebnis (digitale Signatur) wird in Form einer JSON Web Signature (JWS, [RFC 7517: JSON Web Key \(JWK\)](#)) im Feld **signatur** übertragen:

```
BASE64(Header) || '.' || BASE64(Hash) || '.' || BASE64(Signatur)
```

Prüfung der Signaturen

Um die übermittelte JWS zu prüfen, MUSS das empfangende System wiederum

- den Hashwert der empfangenen Nachricht berechnen
- die übermittelte Signatur auf Gültigkeit prüfen
- den signierten Hashwert (JWS-Payload) auf Gleichheit mit dem Hashwert der empfangenen Nachrichten prüfen

Die Ermittlung des Hashwertes erfolgt gemäß "Hashing der Nachrichten".

Die Signatur wird aus der empfangenen Nachricht extrahiert und gegen den im KT-Register hinterlegten öffentlichen Schlüssel des Senders validiert.

Das Signieren von Nachrichten ist optional. Das KT-Register beinhaltet im Feld **transmitsUnsignedMessages** Information, ob ein KT signierte oder nicht-signierte Meldungen sendet.

3.4.2. Verschlüsselung

Da jegliche Kommunikation zwischen KT und UCRM und zwischen UCRMs TLS-verschlüsselt stattfindet, kann auf eine E2E-Verschlüsselung des Nachrichteninhaltes verzichtet werden. Um die strenger Sicherheitsanforderungen vor allem bei der Kommunikation über das Internet zu erfüllen, kann es später jedoch sinnvoll sein, den Nachrichteninhalt zusätzlich Ende-Zu-Ende, d.h. zwischen dem Sender-UCRM und dem Empfänger-UCRM oder sogar zwischen den KTs (in diesem Fall ohne Möglichkeit, die Meldungen durch die UCRM validieren zu lassen), zu verschlüsseln.

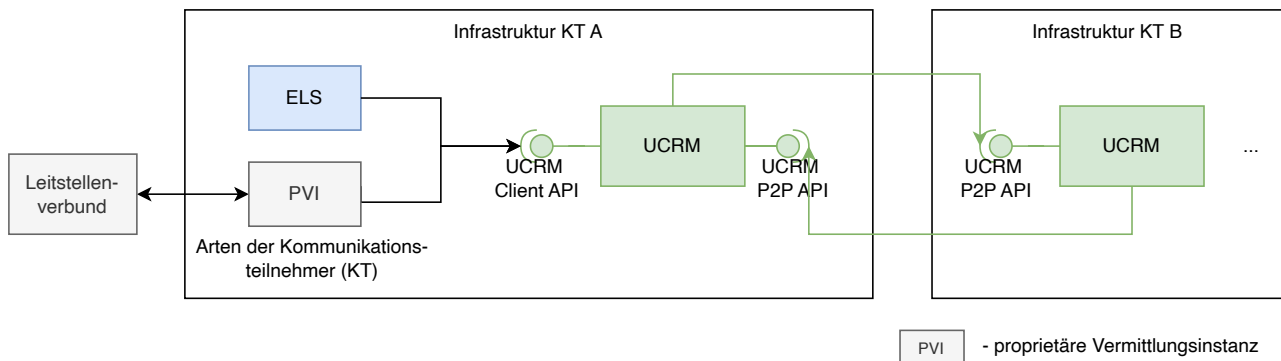
Für die Verschlüsselung wird dann das RSA-Verfahren RSASSA-PKCS1-v1_5 ([RFC 3447: Public-Key Cryptography Standards \(PKCS\) #1: RSA Cryptography Specifications Version 2.1](#)) verwendet.

Das konkrete Verschlüsselungsverfahren wird bei Bedarf in einer späteren Version des UCRI2-Standards spezifiziert.

4. API

In diesem Kapitel wird die konkrete Umsetzung der UCRI2-APIs beschrieben. Gemäß dem Rollenkonzept werden zwei APIs unterschieden, siehe Abbildung:

- Die **Client-API** wird vom **UCRM** angeboten und von **Clients** (Kommunikationsteilnehmern) konsumiert.
- Die **P2P-API** wird von **UCRMs** angeboten, welche eine Verbindung mit anderen **UCRMs** auf Basis einer Peer-to-Peer-Topologie herstellen wollen.



Für beide APIs gelten folgende allgemeinen Festlegungen, welche in den jeweiligen Unterkapiteln mit API-spezifischen Festlegungen ergänzt werden:

1. Das grundlegende API-Design verwendet REST API Design Richtlinien erarbeitet bei [TM Forum](#).
2. Die konkrete Ausgestaltung der APIs inklusive der zu verwendenden Endpunkte und Datenformate wird in den zu der Spezifikation gehörenden OpenAPI-Spezifikationen in Version 3.1.0 (festgelegt in [OpenAPI Initiative](#)) festgelegt. Die OpenAPI-Spezifikationen sind strukturiert aufgebaut. Die übergeordneten API-Beschreibungen (`ucrm-client.yaml` sowie `ucrm-p2p.yaml`) verweisen jeweils auf die im Unterverzeichnis "schemas" vorhandenen `yaml`-Dateien mit den zu übertragenden Datenobjekten. Zusätzlich existieren noch inhaltsgleiche, gebündelte OpenAPI-Spezifikationen im JSON Format (`ucrm-client-bundled.json` sowie `ucrm-p2p-bundled.json`).
3. Beide APIs sind in folgende Funktionsbereiche aufgeteilt:
 - Authentifizierung per OAuth 2.0 Client Credential Grant Flow.
 - KT-Registry - Abfragen von Eigenschaften der registrierten Kommunikationsteilnehmer.
 - Messaging - Versenden und Empfangen von Nachrichten.
 - Info - Informationen über die Version und den Betreiber der Schnittstelle.
4. Die Datenübertragung erfolgt ausschliesslich per Transportverschlüsselung (TLS).
5. Die zur Übertragung von Fehlerzuständen genutzten Error-Objekte (vgl. `error.yaml`) werden im Unterkapitel [Fehlerbehandlung](#) für beide APIs gemeinsam dargestellt, da ein Großteil der genutzten Fehlercodes in beiden APIs vorkommt.
6. Die Generierung und Prüfung von Nachrichtensignaturen wird im Kapitel [Kommunikationsprotokoll](#) im Unterkapitel [Datenintegrität](#) beschrieben.
7. Falls ein Request nicht der OpenAPI-Spezifikation entspricht, MUSS dieser vom UCRM zurückgewiesen werden (vgl. Kapitel [Fehlerbehandlung](#)).
8. Ein UCRM, welches die **Client-API** implementiert, MUSS diese vollständig unterstützen.
9. Ein UCRM, welches die **P2P-API** implementiert, MUSS diese vollständig unterstützen.

4.1. Client-API

Die Client-API dient der Anbindung von KT (clients) an die Transportschicht (UCRM). Im Folgenden werden die API-Endpunkte vorgestellt:

4.1.1. /token - HTTP GET

Über den Token-Endpunkt werden JSON Web Token (JWT) abgerufen, die bei sämtlichen anderen Endpunkten zur Autorisierung genutzt werden. Hierzu werden eine Nutzerkennung sowie ein Geheimnis per HTTP-Header übergeben. Hierbei sei angemerkt, dass gemäß RFC 7519 ein **exp**-Claim als NumericDate dargestellt wird (also als die Sekunden seit 1970-01-01T00:00:00Z UTC), vgl. RFC 7519 Kapitel 2.

4.1.2. /info - HTTP GET

Über den Info-Endpunkt können allgemeine Informationen über das UCRM wie die genutzte UCRI2-Transportschicht-Version, UCRM-Hersteller, Produktversion und Betriebsstatus des UCRM abgerufen werden.

4.1.3. /registry - HTTP GET

Über den Registry-Endpunkt können KT Informationen über alle dem UCRM bekannten Kommunikationsteilnehmer (inklusive den UCRMs selbst) abrufen. Die Datenfelder verändern sich hierbei im Normalfall nur selten, eine Ausnahme bildet das **status**-Feld. Dieses gibt für einen KT an, ob dieser aktuell erreichbar ist. Der Wert dieses Feldes verändert sich je nach von der Vermittlungsebene bzw. den beteiligten UCRMs festgestellten Verbindungsstatus relativ dynamisch (vgl. [Verfügbarkeitsstatus unter API-Endpunkt /messaging/receive](#)). Da die Umgebung dynamisch ist, d.h. neue KT können dazukommen, existierende KT können abgebaut werden, sollen KT-Register-Abfragen regelmäßig durchgeführt werden (mindestens 1 Mal pro Stunde, maximal alle 5 Minuten).

4.1.4. /registry/{id} - HTTP GET

Über den Registry-ID-Endpunkt können KT Informationen über einen konkreten Kommunikationsteilnehmer abrufen, welcher die übergebene OID besitzt.

4.1.5. /messaging/send - HTTP POST

Über den Messaging-Send-Endpunkt können Nachrichten an andere KT versendet werden. Hierbei werden die Anwendungsnachrichten in eine sog. **Envelope** verpackt, welche Transportschicht-Metadaten enthält. Hierbei MUSS das UCRM folgende Prüfungen durchführen und im Fehlerfall einen passendes Error-Objekt zurückgeben:

1. Prüfung, ob der KT zum Senden der Nachricht berechtigt ist (Abgleich des source-Feldes mit dem übergebenen OAuth-Token).
2. Prüfung, ob der adressierte KT (destinations-Feld) bekannt ist.
3. Prüfung, ob die enthaltene Anwendungsnachricht (payload-Feld) eine bekannte Anwendung in einer bekannten Anwendungsversion referenziert und gemäß des Nachrichtenschemas gültig ist.
4. Prüfung, ob der adressierte KT (destinations-Feld) den Empfang der Anwendungsnachricht gemäß dessen KT-Registrierungs-Eintrages unterstützt.

Die Prüfung von Nachrichtensignaturen wird durch das UCRM für von clients gesendete Nachrichten NICHT durchgeführt, dies obliegt dem Empfänger der Nachricht.

Falls das UCRM eine unbekannte destination meldet, sollte der client über den /info-Endpunkt prüfen, ob das UCRM sich in einem normalen Betriebszustand befindet oder aktuell den initialen Abruf der KT-Registrierungsdaten von verbundenen UCRMs durchführt. In diesem Falle sollte abgewartet werden, bis das UCRM einen normalen Betriebszustand meldet.

Zustellungsquittierungen

Um die verlässliche Zustellung sicherzustellen, kann ein Client sowohl einen timeout-Wert für die Zustellung der Nachricht (**timeout**-Feld) setzen als auch eine explizite Empfangsbestätigung (**ack**-Feld) anfragen. In diesen Fällen überträgt das UCRM dem Client entweder eine negative Quittierung (falls ein Timeout aufgetreten ist oder bei der Weiterleitung der Nachricht innerhalb der Transportschicht ein Fehler aufgetreten ist) oder eine positive Quittierung (falls dies durch das **ack**-Feld angefragt wurde und der Empfänger den Empfang der Nachricht per Messaging-Commit-Endpunkt bestätigt hat). Hierzu kommt die **message_delivery_status**-Nachricht aus der **transport-layer-messages**-App zum Einsatz. Bezüglich dieser Statusbenachrichtigungen gelten folgende Festlegungen:

1. **message_delivery_status**-Nachrichten dürfen NUR durch UCRMs versendet werden, nicht durch clients. Dies muss durch das UCRM sichergestellt werden.
2. Ein UCRM sollte für eine versendete Nachricht MAXIMAL eine **message_delivery_status**-Nachricht an den KT zurückmelden. Da aber die Übermittlung der positiven Zustellungsquittierung an das sendende UCRM sich mit der

Generierung des Timeouts überschneiden kann, ist es dennoch möglich, dass ein KT mehrere **message_delivery_status**-Nachrichten erhält.

3. Die Zuständigkeit für den Versand einer negativen Timeout-Quittierungs-Nachricht liegt IMMER bei dem UCRM, an welches der Sender angebunden ist. So wird sichergestellt, dass Timeout-Nachrichten auch dann generiert werden, wenn Übermittlungs-Probleme innerhalb der Transportschicht auftreten.

Wiederholter Nachrichtenversand

Falls ein KT eine Nachricht erneut versenden will (z.B. weil für den ersten Sendungsversuch ein Timeout gemeldet wurde), MUSS für die erneut versendete Nachricht die gleiche **messageld** verwendet werden. So wird sichergestellt, dass die duplizierte Nachricht anhand der **messageld** durch den empfangenden KT ausgefiltert werden kann.

4.1.6. /messaging/receive - HTTP POST

Über den Messaging-Receive-Endpunkt können Nachrichten, die an den KT (bzw. eine Liste von OIDs im Feld **destinations**) adressiert sind, abgerufen werden. Diese Operation ist idempotent, kann also mehrmals mit dem gleichen Ergebnis wiederholt werden. Um den wiederholten Empfang einer Nachricht zu verhindern, MUSS der KT vor dem nächsten Abruf von Nachrichten die erfolgreich erhaltenen Nachrichten per Messaging-Commit-Endpunkt bestätigen.

Das UCRM MUSS prüfen, ob der Client zum Abruf der übergebenen OIDs berechtigt ist (durch Abgleich mit dem übergebenen OAuth-Token).

Das UCRM MUSS für jede empfangene Nachricht eine monoton steigende Sequenznummer im Feld **sequenceid** zurückgeben, die sich für eine individuelle Nachricht bei eventuellen mehrfachen Abrufen NICHT verändert.

Long-Polling

Da der Messaging-Receive-Endpunkt periodisch abgerufen werden muss, ist für einen schonenden Umgang mit Netzwerkressourcen und zur Sicherstellung möglichst geringer Zustellungsverzögerungen für diesen Endpunkt ein sog. [Long Polling](#) vorgesehen. Dieses MUSS vom UCRM unterstützt werden. Folgende Schleife ist bei Nachrichtenabfragen mittels Long Polling zu verwenden:

1. Nachrichten abfragen mit Angabe maximaler Nachrichtenzahl nMax (Feld **maxMessages**)
2. Falls Nachrichten gemeldet wurden, Nachrichten verarbeiten
3. Falls Nachrichten gemeldet wurden, Nachrichtenempfang bestätigen

Wichtig: bei Programmausfällen zwischen Schritten 2 und 3 kann es zum wiederholten Empfang von gleichen Nachrichten kommen. Die Client-Logik MUSS somit mit Nachrichtenduplikaten umgehen können. Dies kann durch Berücksichtigung der Eindeutigkeit der Nachrichten-ID (Feld **messageld**) geschehen.

Durch die Verwendung von Long Polling kann direkt ohne Pause von Schritt 3 zu Schritt 1 übergegangen werden. Für das Long Polling gelten folgende Festlegungen:

1. Die maximale Verzögerung (dMax) der Antwort beträgt 30 Sekunden.
2. Falls für mindestens eines der angefragten Ziele (Feld **destinations**) mindestens eine unbestätigte Nachricht vorliegt, wird die Anfrage sofort beantwortet.
3. Ansonsten wird die Beantwortung der Client-Anfrage solange verzögert, bis eine der folgenden Bedingungen vorliegt (die Bedingungen sind in der genannten Reihenfolge zu prüfen), dann wird die Anfrage ohne weitere Verzögerung beantwortet:
 1. Für mindestens eines der angefragten Ziele (Feld **destinations**) liegt mindestens eine unbestätigte Nachricht vor.
 2. Die maximale Verzögerung (dMax) der Antwort wurde erreicht.

4. Falls beim Long Polling Probleme auftreten, die sich nicht auf anderem Wege lösen lassen (z.B. durch die clientseitige Verlängerung von TCP-Timeouts), kann der Client mittels des Feldes **maxDelay** ein verkürztes dMax angeben, welches dann vom UCRM anstelle des in 1. genannten dMax zu verwenden ist.

Verfügbarkeitsstatus

Der periodische Abruf des Messaging-Receive-Endpunkts dient dem UCRM zusätzlich als Indikator über die aktuelle Erreichbarkeit des Client. Falls ein Client den Messaging-Send-Endpunkt für 60 Sekunden (Verfügbarkeits-Timeout) nicht abrufen, MUSS das UCRM den KT-Registrierungs-Verfügbarkeitsstatus (Feld **status**) auf "offline" setzen. Sobald wieder eine Anfrage eingeht, MUSS das UCRM den Verfügbarkeitsstatus auf "online" setzen. So können andere KT den aktuellen Verfügbarkeitsstatus via KT-Registrierung abrufen. Durch die Wahl des Verfügbarkeitsstatus-Timeout (60 Sekunden) auf das Doppelte der maximalen Long-Polling-Verzögerung (30 Sekunden) ist sichergestellt, dass beide Funktionen miteinander harmonisieren. Um aus Sicht der Vermittlungsebene als verfügbar zu erscheinen, MUSS ein Client also eine permanente Abfrage von Nachrichten durchführen (vgl. die im Kapitel "Long Polling" dargestellte Schleife).

4.1.7. /messaging/commit - HTTP POST

Über den Messaging-Commit-Endpunkt kann der erfolgreiche Empfang von Nachrichten für eine einzelne destination bestätigt werden. Hierzu wird eine Referenz-Sequenznummer angegeben (Feld **sequenceId**). Das UCRM entfernt dann alle Nachrichten mit Nachrichten-Sequenznummer $\leq$ Referenz-Sequenznummer aus der Liste unbestätigter Nachrichten und löst je nach Konfiguration des **ack**-Feldes die Übersendung einer Zustellbestätigung (**message-delivery-status**) aus. Falls ein Client Nachrichten für verschiedene OIDs abrufen, MUSS für jede Nachricht ein separater Aufruf des Messaging-Commit-Endpunktes durchgeführt werden.

Diese Operation ist idempotent, kann also mehrmals mit dem gleichen Ergebnis wiederholt werden.

Das UCRM MUSS prüfen, ob der Client zur Bestätigung von Nachrichten an die übergebenen OIDs (Feld **destination**) berechtigt ist (durch Abgleich mit dem übergebenen OAuth-Token).

4.2. Peer-to-Peer-API (P2P-API)

Die P2P-API dient der Vernetzung innerhalb der Transportschicht, also von **UCRMs** untereinander. Jedes beteiligtes UCRM bietet die API einerseits selbst an und verbindet sich andererseits mit den von entfernten UCRMs angebotenen APIs. Im Folgenden werden die API-Endpunkte vorgestellt:

4.2.1. /token - HTTP GET

Über den Token-Endpunkt werden JSON Web Token (JWT) abgerufen, die bei sämtlichen anderen Endpunkten zur Authorisierung genutzt werden. Hierzu werden eine Nutzerkennung sowie ein Geheimnis per HTTP-Header übergeben. Hierbei sei angemerkt, dass gemäß RFC 7519 ein **exp**-Claim als NumericDate dargestellt wird (also als die Sekunden seit 1970-01-01T00:00:00Z UTC), vgl. RFC 7519 Kapitel 2.

4.2.2. /info - HTTP GET

Über den Info-Endpunkt können allgemeine Informationen über das UCRM wie die genutzte UCRI2-Transportschicht-Version, UCRM-Hersteller, Produktversion und Betriebsstatus des UCRM abgerufen werden.

4.2.3. /registry - HTTP GET

Über den Registry-Endpunkt können entfernte UCRMs Informationen über alle direkt an das angerufene UCRM angebotenen Kommunikationsteilnehmer (inklusive den UCRMs selbst) abrufen. Hierbei MUSS das UCRM ausschliesslich die lokalen KT zurückgeben, nicht jedoch weitere über entfernte UCRMs angebundene KT.

Falls einzelne übertragenen KT-Registrierungsdatensätze fehlerhaft sind, MUSS das UCRM diese verwerfen.

Da die Umgebung dynamisch ist, d.h. neue KT können dazukommen, existierende KT können abgebaut werden, sollen KT-Register-Abfragen regelmäßig durchgeführt werden (mindestens 1 Mal pro Stunde, maximal alle 5 Minuten). Außerdem sollte ein UCRM beim Start die KT-Registrierungsdaten aller bekannten anderen UCRMs abrufen. Ein UCRM MUSS während dieser Phase den am Info-Endpunkt zurückgegebenen Status-Wert auf den Status 1 setzen (sowohl für die P2P- als auch für die Client-API).

Direkter Versand von Verfügbarkeits-Status-Nachrichten

Falls ein UCRM feststellt, dass sich der Verfügbarkeitsstatus eines Clients verändert hat, wird das KT-Registrierungs-**Status**-Feld aktualisierung (vgl. Unterkapitel "Client-API/Verfügbarkeitsstatus"). Diese Änderung wird dann entfernten UCRMs erst dann bekannt, sobald diese den nächsten periodischen Abruf der KT-Registrierung durchführen. Um die Statusveränderung direkt und ohne Verzögerung an alle entfernten UCRMs zu kommunizieren, MUSS das UCRM daher zusätzlich nach Feststellung der Statusänderung eine **participant_availability_update**-Nachricht (aus der **transport_layer_messages**-App) an alle entfernten UCRMs senden.

Falls ein UCRM eine solche **participant_availability_update**-Nachricht erhält, MUSS es den KT-Registrierungs-Status des genannten KT auf den übermittelten Wert anpassen. Die Nachricht selbst darf NICHT an die angebundenen Clients weitergegeben werden.

4.2.4. /messaging/send - HTTP POST

Über den Messaging-Send-Endpunkt werden Nachrichten an entfernte UCRM weitergegeben. Hierbei gelten folgende Festlegungen:

1. Falls das entfernte UCRM die Nachrichtenannahme ablehnt und im **ack**-Feld nicht "none" angegeben war, MUSS das sendende UCRM dem KT, welcher die Nachricht ursprünglich gesendet hat, eine negative Zustellungs-Status-Nachricht übersenden, in deren **cause**-Feld der gemeldete Fehler mitübertragen wird.
2. Das empfangende UCRM MUSS prüfen, ob die Nachricht an einen der am UCRM angebundenen KT oder das UCRM selbst adressiert ist, falls dies nicht der Fall ist, MUSS die Nachricht abgelehnt werden.
3. Das empfangende UCRM MUSS prüfen, ob die enthaltene Anwendungsnachricht (payload-Feld) eine bekannte Anwendung in einer bekannten Anwendungsversion referenziert und gemäß des Nachrichtenschemas gültig ist.
4. Das empfangende UCRM MUSS prüfen, ob der adressierte KT (destinations-Feld) den Empfang der Anwendungsnachricht gemäß dessen KT-Registrierungs-Eintrages unterstützt.

4.3. Fehlerbehandlung

Falls als Antwort auf eine Anfrage ein Fehler zurückgegeben wird, enthält dieser IMMER einen numerischen, UCRI2-spezifischen Fehlercode. Dieser Fehlercode ermöglicht es, die Fehlerursache zusätzlich zur groben Fehlereinteilung per HTTP-Statuscode genauer festzustellen. Für jeden Fehlercode wird eine feste Fehlercode-ID vergeben, z.B. für den Fehlercode 460 die Fehlercode-ID REQUEST_INVALID_PER_CLIENT_TRANSPORT_SPEC (s.u.).

Andere als die unten genannten Fehlercodes dürfen NICHT zurückgegeben werden.

Falls dies nicht explizit angegeben ist, können alle Fehlercodes sowohl von der Client- als auch von der P2P-Schnittstelle zurückgegeben werden.

Zusätzlich vom Fehlercode wird IMMER eine menschenlesbare Fehlermeldung im reason-Feld mitübergeben. Die zusätzliche Übermittlung einer detaillierten Fehlerursache im message-Feld ist dagegen freiwillig.

Für jeden Fehlercode wird an dieser Stelle dargestellt, in welchen Situationen dieser Fehlercode zurückgegeben wird. Die Fehlercodes sind dabei nach den HTTP-Statuscodes geordnet, in deren Kontext sie auftreten. Fehlercodes dürfen NUR im Kontext des aufgeführten HTTP-Statuscodes übertragen werden. Eine Ausnahme bildet hierbei die Übertragung eines NAK-Zustellfehlers im Rahmen der message_delivery_status-Nachricht in der transport_layer_messages-App, wo der Fehlercode, der von einem entfernten UCRM über das P2P-Schnittstelle gemeldet wurde an den Client weitergeleitet wird.

4.3.1. Fehlercodes für Antworten mit HTTP-Statuscode 400

- **460 (REQUEST_INVALID_PER_CLIENT_TRANSPORT_SPEC)** : Dieser Fehlercode wird NUR von der Client-Schnittstelle zurückgegeben, falls der Request zwar valide JSON-Daten enthält, diese aber das Client-Transportschicht-Schema für diesen Request verletzen. Der Fehlercode kommt NICHT zum Einsatz, wenn ein enthaltener App-Payload das App-Schema verletzt (vgl. **REQUEST_PAYLOAD_INVALID_PER_APP_SPEC**).
- **461 (REQUEST_PAYLOAD_UNKNOWN_APPID)** : Dieser Fehlercode wird zurückgegeben, falls die angegebene AppId dem UCRM unbekannt ist.
- **462 (REQUEST_PAYLOAD_UNKNOWN_APPVERSION)** : Dieser Fehlercode wird zurückgegeben, falls dem UCRM zwar die AppId bekannt ist, aber die angegebene AppVersion unbekannt ist.
- **463 (REQUEST_PAYLOAD_UNKNOWN_SCHEMAID)** : Dieser Fehlercode wird zurückgegeben, falls dem UCRM zwar die AppId und AppVersion bekannt ist, aber die angegebene SchemaId (also der Nachrichtentyp) unbekannt ist.
- **464 (REQUEST_PAYLOAD_INVALID_PER_APP_SPEC)** : Dieser Fehlercode wird zurückgegeben, falls der Request-Payload im data-Feld das zugehörige App-Schema verletzt.
- **465 (REQUEST_PAYLOAD_INVALID_JSON)** : Dieser Fehlercode wird zurückgegeben, falls es sich bei den übergebenen Daten nicht um gültiges JSON handelt.
- **466 (REQUEST_PAYLOAD_UNSUPPORTED_APPID_OR_APPVERSION)** : Dieser Fehlercode wird zurückgegeben, falls die appId und appVersion dem UCRM zwar bekannt, aber nicht in den supportedApps der destination als unterstützt aufgeführt sind.
- **468 (REQUEST_PAYLOAD_UNSUPPORTED_MESSAGE)** : Dieser Fehlercode wird zurückgegeben, falls die appId und appVersion dem UCRM bekannt und auch in den supportedApps der destination als unterstützt aufgeführt sind, die schemaId (also der Nachrichtentyp) aber im entsprechenden supportedApps-Eintrag in der liste der unsupportedMessages aufgeführt ist.
- **467 (REQUEST_PAYLOAD_FORBIDDEN_APPID)** : Dieser Fehlercode wird NUR von der Client-Schnittstelle zurückgegeben, falls die gesendete Nachricht zur transport_layer_messages-App gehört (Nachrichten dieser App dürfen nur durch UCRMs versendet werden).
- **470 (REQUEST_UNKNOWN_DESTINATION_ID)** : Dieser Fehlercode wird zurückgegeben, wenn die angegebene destination dem UCRM nicht bekannt ist.
- **478 (REQUEST_OID_FORBIDDEN)** : Dieser Fehlercode wird zurückgegeben, falls der durch das übergebene OAuth-Token identifizierte Benutzer keinen Zugriff auf die angegebene OID hat. Bei der angegebenen OID handelt es sich um die OID:
 - im sender-Feld (für den /messaging/sent-Endpunkt)
 - im destinations-Feld (für den /messaging/receive-Endpunkt)
 - im destination-Feld (für den /messaging/commit-Endpunkt)
- **479 (REQUEST_WRONG_SIGNATURE)** : Dieser Fehlercode wird NUR von der P2P-Schnittstelle zurückgegeben, falls es sich bei der gesendeten Nachricht um eine signierte Nachricht aus der transport_layer_messages-App handelt (und das sendende UCRM nicht per transmitsUnsignedMessages in seinem KT-Registrierungseintrag festgelegt hat, dass dieses seine Nachrichten nicht signiert). Auf der Client-Schnittstelle und generell für von Clients gesendete Nachrichten findet KEINE Signaturprüfung in der Transportschicht statt.
- **480 (REQUEST_INVALID_PER_P2P_TRANSPORT_SPEC)** : Dieser Fehlercode wird NUR von der P2P-Schnittstelle zurückgegeben, falls der Request zwar valide JSON-Daten enthält, diese aber das P2P-Transportschicht-Schema

für diesen Request verletzen. Der Fehlercode kommt NICHT zum Einsatz, wenn ein enthaltener App-Payload das App-Schema verletzt (vgl. REQUEST_PAYLOAD_INVALID_PER_APP_SPEC).

4.3.2. Fehlercodes für Antworten mit HTTP-Statuscode 401

- **475 (REQUEST_UNAUTHORIZED)** : Dieser Fehlercode wird zurückgegeben, falls das übergebene OAuth-Token fehlt, ungültig oder expired ist. Außerdem wird der Fehlercode durch den /token-Endpunkt zurückgegeben, falls die übergebenen Zugangsdaten nicht korrekt sind.

4.3.3. Fehlercodes für Antworten mit HTTP-Statuscode 500

- **491 (REQUEST_INTERNAL_ERROR)** : Dieser Fehlercode wird zurückgegeben, falls ein interner Fehler aufgetreten ist.

5. Applikationen

Die konkreten UCRI2-App-Spezifikationen werden separat zur Verfügung gestellt. Hierbei gibt das Dokument "UCRI2-Apps im Überblick" einen Überblick über die verschiedenen Apps.

Zusätzlich existieren für jede App in jeder Version jeweils eine PDF-Datei mit der textuellen App-Dokumentation sowie pro App-Nachricht jeweils eine JSON-Schema-Datei, welche die Nachrichtenstruktur spezifiziert.

6. Systemintegration

Die UCRI2-Spezifikation bietet einen Standardrahmen für die Integration von Leitstellen-Software unterschiedlicher Hersteller. Sie hilft damit, Kompatibilitätsprobleme zu überwinden und dadurch die Interoperabilität in komplexen IT-Landschaften sicherzustellen. Gleichzeitig bleibt die Vermittlungsebene offen für nahtlose Integration von bestehenden Systemlösungen in neue Kommunikationssysteme. Das ermöglicht, bereits vorhandene Infrastrukturen weiterzuverwenden und so Entwicklungskosten und -zeiten zu reduzieren. Dadurch wird eine nachhaltige und effiziente Systemlandschaft geschaffen, die sowohl Innovation als auch Kontinuität gewährleistet.

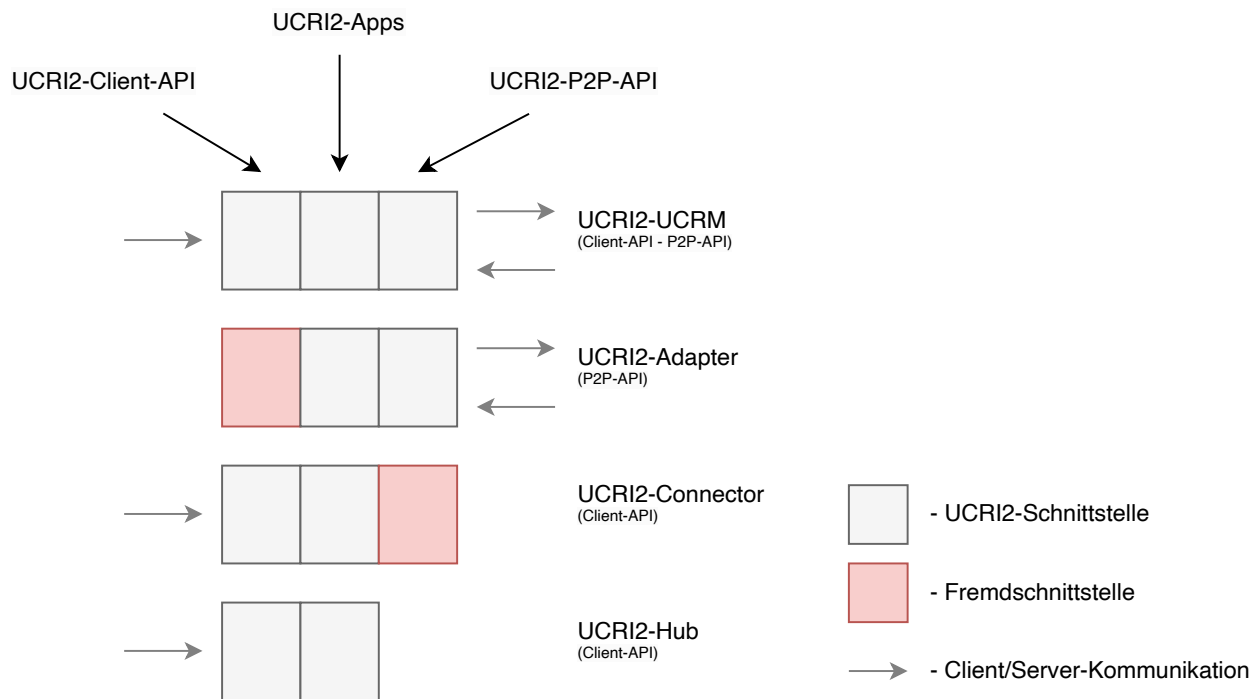
In diesem Kapitel werden Integrationsmuster und dazugehörige Integrationsszenarien dargestellt.

6.1. Integrationsmuster

Das zentrale Element der UCRI2-Spezifikation ist das UCRM – eine Systemkomponente, die durch folgende Schnittstellen beschrieben wird:

- Die **Client-API** wird vom **UCRM** angeboten und von **Clients** (Kommunikationsteilnehmern) konsumiert.
- Die **P2P-API** wird von **UCRMs** angeboten, welche eine Verbindung mit anderen **UCRMs** auf Basis einer Peer-to-Peer-Topologie herstellen wollen.

Je nach Systemlandschaft brauchen Systemintegratoren spezialisierte UCRM-Module, die unterschiedliche Integrationsszenarien unterstützen:



UCRI2-UCRM

Das klassische UCRI2-UCRM-Modul ist eine universelle Komponente, die als Vermittler in einem Standard-UCRI2-Verbund eingesetzt wird. Das Modul verbindet eine oder mehrere über die UCRI2 Client-API angebundene Leitstellen mit anderen Leitstellen, die über ein fremdes UCRI2-UCRM-Modul angebunden sind. Zwischen UCRMs wird das UCRI2 P2P-Protokoll verwendet, um die Nachrichten zu übermitteln und andere UCRI2-spezifische Daten auszutauschen.

UCRI2-Adapter

Der UCRI2-Adapter dient dazu, ein bestehendes Leitstellenkommunikationssystem in ein UCRI2-Verbund zu integrieren. Das Modul stellt dabei eine Schnittstelle bereit, die das bestehende System erwartet. Der UCRI2-Adapter fungiert dabei als Vermittler, der ein Fremdkommunikationsprotokoll inklusive Mapping zwischen Adressierungsschemata in das UCRI2 P2P-Protokoll übersetzt.

UCRI2-Connector

Der UCRI2-Connector verbindet eine über UCRI2 Client-API angebundene Leitstelle mit einem fremden Nachrichtenvermittlungssystem. Der UCRI2-Connector kapselt dabei spezifische fremde Protokolle, Datenflüsse und Kommunikationsmechanismen inklusive Mapping zwischen Adressierungsschemata, um eine Integration über ein bestehendes fremdes Vermittlungssystem zu ermöglichen.

UCRI2-Hub

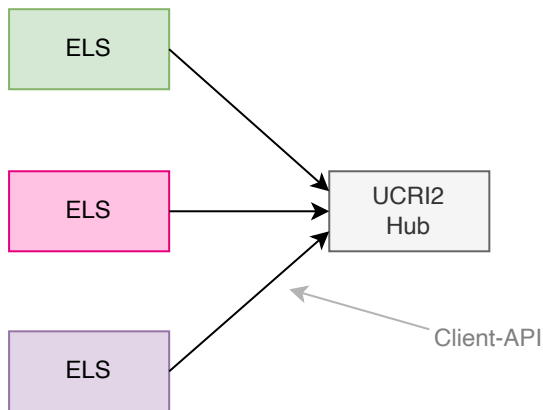
Der UCRI2-Hub dient als eine zentrale Komponente, die als Vermittler zwischen mehreren über UCRI2 Client-API angebotenen Leitstellen fungiert. Alle Nachrichten fließen über diesen zentralen Punkt, wodurch die Kommunikation zwischen den Teilnehmern entkoppelt wird. Der UCRI2-Hub unterstützt keine Protokolle der Vermittlungsebene und kann somit nicht mit anderen UCRMs verbunden werden.

6.2. Integrationsszenarien

Im Folgenden werden unterschiedliche Integrationsszenarien unter Verwendung von spezialisierten UCRM-Modulen vorgestellt.

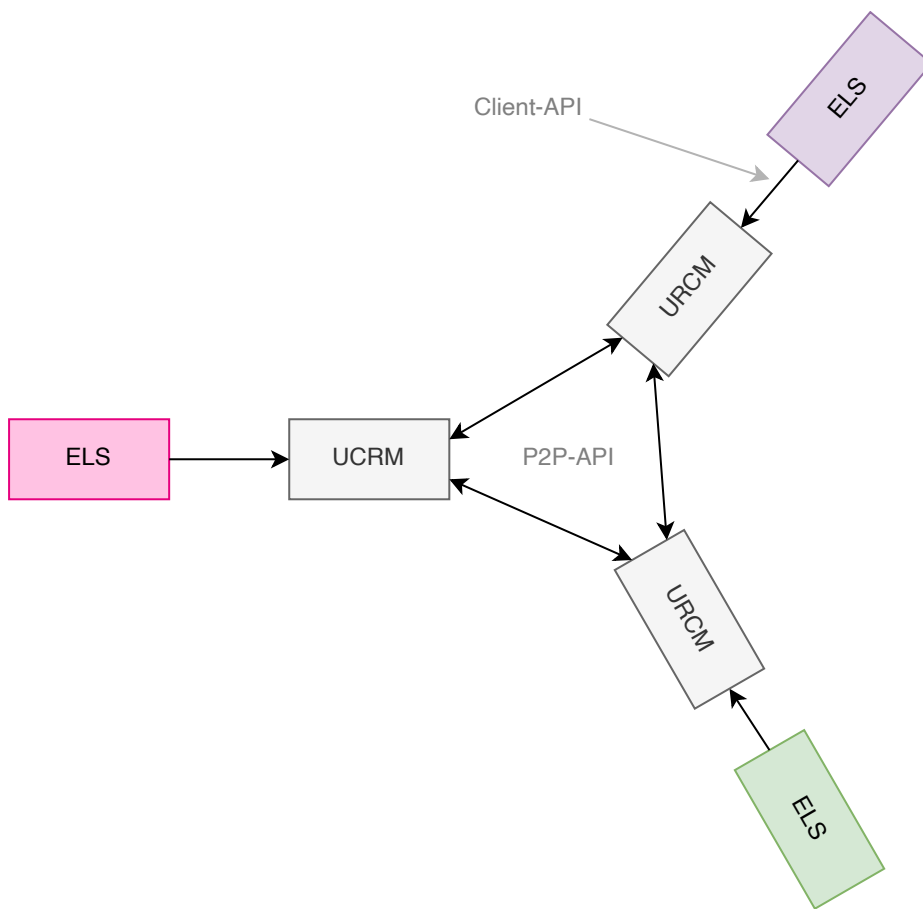
Ein UCRI2-Hub als Vermittler

Im einfachsten Fall werden Leitstellen über einen zentralen UCRI2-Hub verbunden. Die ELS-Software wird über UCRI2 Client-API an dieses Modul angebunden. Somit können entkoppelte ELS miteinander kommunizieren. Der UCRI2-Hub unterstützt keine Protokolle der Vermittlungsebene und kann somit nicht mit anderen UCRMs verbunden werden.



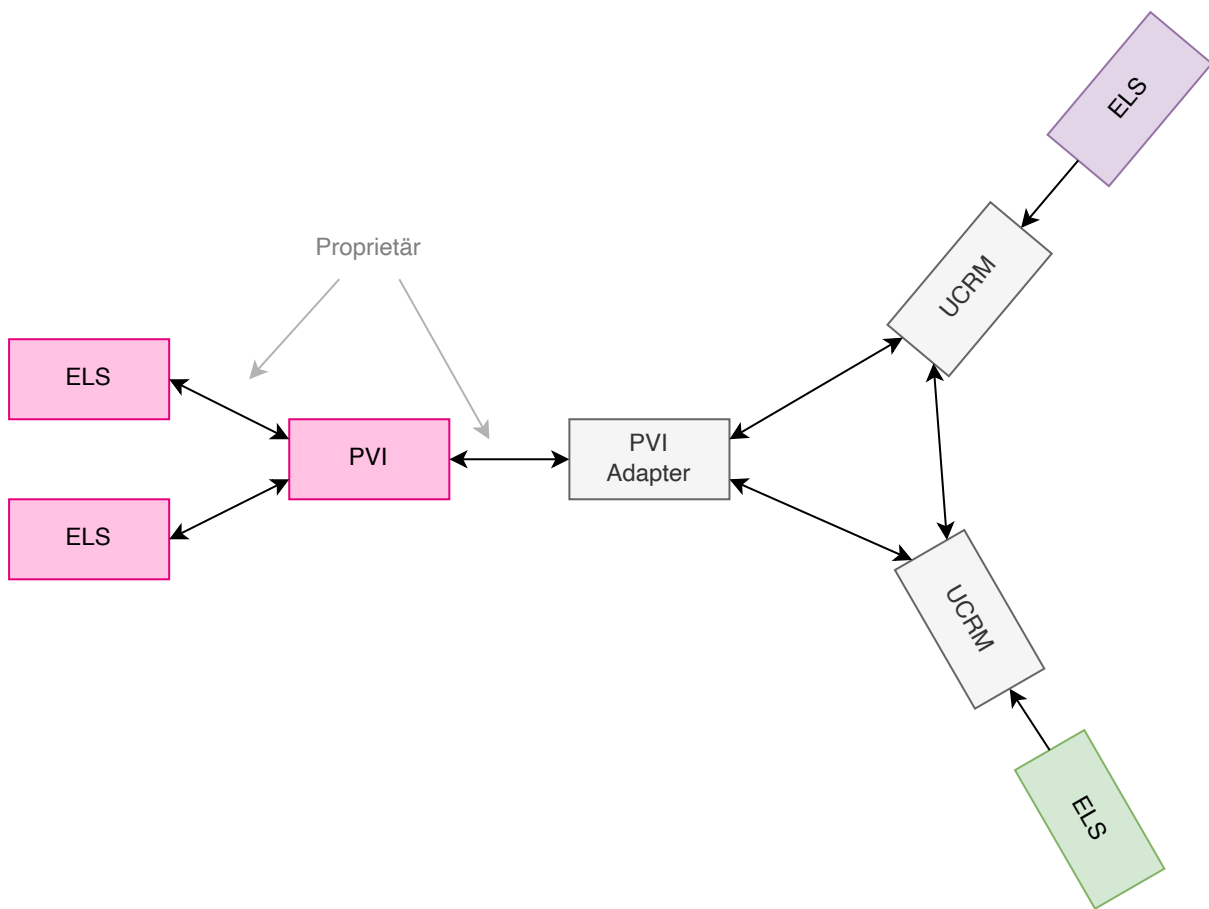
Der Standard-UCRI2-Verbund

In einem Standard-UCRI2-Verbund werden Leitstellen über klassische UCRI2-UCRM-Module verbunden. Typischerweise wird in der Infrastruktur einer Leitstelle ein UCRI2-UCRM-Modul etabliert. Die ELS-Software wird über UCRI2 Client-API an dieses Modul angebunden. Die UCRI2-UCRM-Module unterschiedlicher Kommunikationsteilnehmer werden über UCRI2 P2P-API miteinander verbunden.



Anbindung an Verbund mit PVI

In diesem Szenario wird ein bestehendes Leitstellenkommunikationssystem in ein UCRI2-Verbund aufgenommen. Die Anbindung erfolgt über einen PVI-Adapter – einen spezialisierten UCRI2-Adapter, der die Kommunikationsschnittstelle einer proprietären Vermittlungsinstanz unterstützt. Auf der anderen Seite spricht der PVI-Adapter das UCRI2 P2P-Protokoll und kann somit in die P2P-Topologie des Verbundes aufgenommen werden. Der PVI-Adapter übersetzt das PVI-Kommunikationsprotokoll in das UCRI2 P2P-Protokoll inklusive Mapping zwischen Adressierungsschemata.



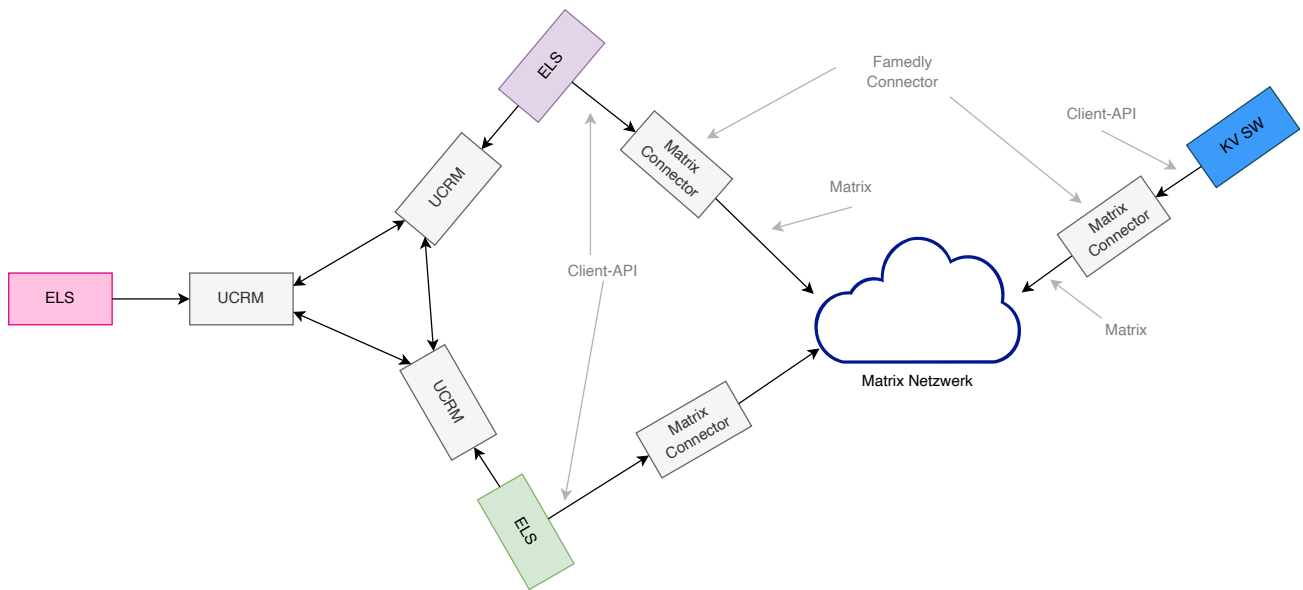
Verbund mit Anbindung an KV-Verbund ohne KV-Adapter

Ein wichtiger Anwendungsfall für die UCRI2-Spezifikation ist Kommunikation zwischen 110/112- und KV-Leitstellen.

Die Ausgangssituation für dieses Integrationsszenario ist ein bestehendes Standard-UCRI2-Verbund und eine Anforderung eine bestimmte Leitstelle aus dem Verbund mit einer KV-Leitstelle zu verbinden. Dabei soll die aktuelle Entwicklung berücksichtigt werden, wonach die KVs über eine normierte Vermittlungsarchitektur auf Basis einer [Matrix-Infrastruktur](#) kommunizieren müssen.

Für die Integration zwischen Leitstellen wird auf beiden Seiten ein Matrix-Connector - ein UCRI2-Connector zur Anbindung an eine Matrix-Infrastruktur - eingesetzt. Sowohl ELS als auch KV-Software verwenden dabei die UCRI2 Client-API des eigenen Matrix-Connectors. Die Nachrichtenübermittlung erfolgt dabei über das Matrix-Protokoll. Der Matrix-Connector unterstützt das Mapping zwischen UCRI2- und Matrix-Adressierungsschemata.

Ein Problem dieses Integrationsschemas ist, dass jede Leitstelle, die mit KVs integriert werden soll, braucht außer UCRM-Modul zusätzlich einen eigenen Matrix-Connector. Die Lösung dieses Problems besteht in der Anbindung des gesamten UCRI2-Verbundes an die Matrix-Infrastruktur, siehe nächstes Integrationsszenario.



Verbund mit Anbindung an KV-Verbund mit KV-Adapter

In diesem Szenario wird der Gesamt-UCRI2-Verbund an die Matrix-Infrastruktur mit Hilfe eines Matrix-Adapters angebunden. Somit können alle an dem Verbund beteiligten ELS mit KVs kommunizieren ohne einen eigenen Matrix-Connector einsetzen zu müssen.

Der Matrix-Adapter unterstützt das UCRI2 P2P-Protokoll und wird als Teil eines bestehenden UCRI2 P2P-Netzes etabliert. Auf der anderen Seite unterstützt der Matrix-Adapter das Matrix-Protokoll und kann somit mit UCRI2 Matrix-Connectors anderer Teilnehmer einer Matrix-Infrastruktur kommunizieren. Auf dieser Weise werden UCRI2-Domäne, die auf unterschiedlichen Vermittlungstechnologien basieren, zu einem übergeordneten UCRI2-Kommunikationsverbund organisiert, in dem alle Teilnehmer untereinander transparent mit Hilfe von standardisierten UCRI2-Anwendungen kommunizieren können.

